

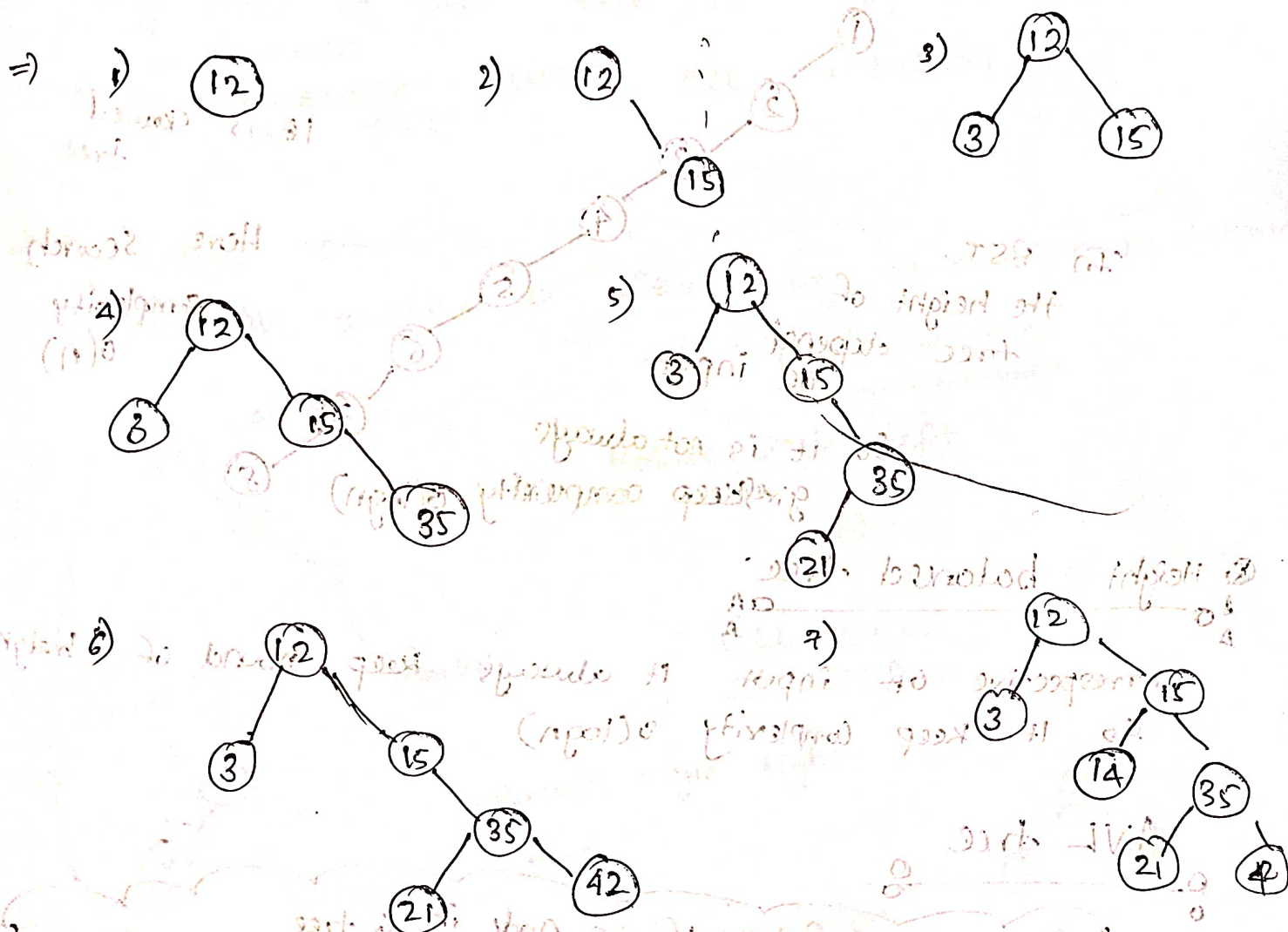
Module - 2



Binary Search tree

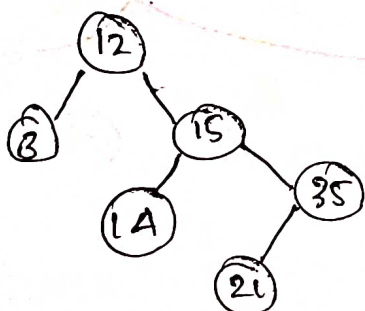
12, 15, 3, 35, 21, 42, 14

construct tree

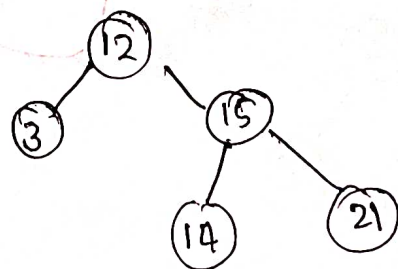


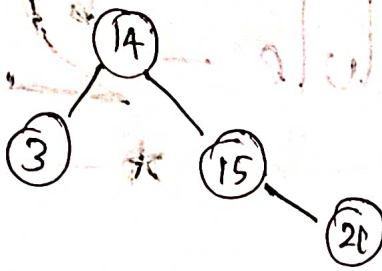
⇒ Delete node 42

i) Case-1



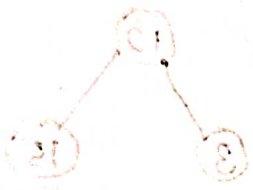
⇒ Delete node 35





replace
inorder successor

Q 1, 2, 3, 4, 5, 6, 7, 8



1

2

3

4

5

6

7

8

It is skewed tree

In BST.

the height of tree depends on input

So it is not always gives complexity $O(\log n)$

Here search complexity $O(n)$

Height balanced tree



Irrespective of input it always keep bound of height
So it keep complexity $O(\log n)$

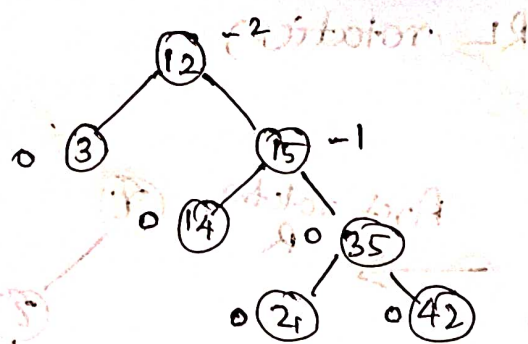
AVL tree



Balancing factor of a node in a tree

height of left subtree - height of right subtree



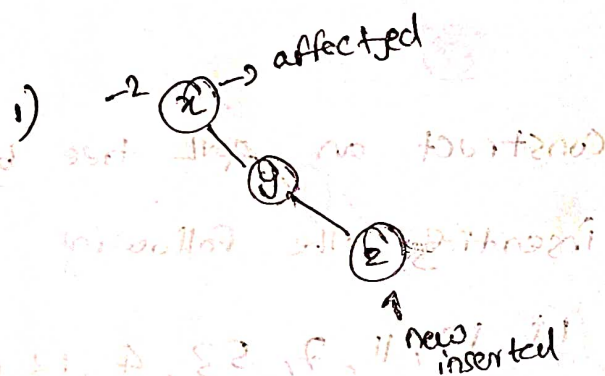


* A tree is said to be AVL if balancing factor of any node in that tree is -1 , 0 , or 1 to maintain

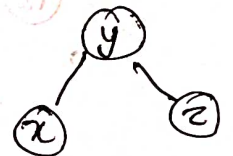
* A Balancing factor using AVL rotation

→ There are 4 AVL rotations

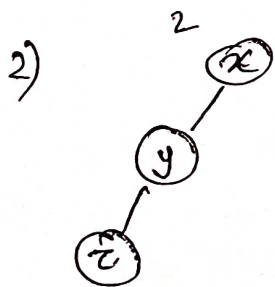
→ Perform AVL rotation after every insertion and deletion



Rotate single left



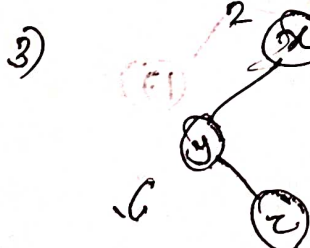
(LL rotation)



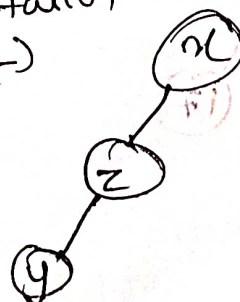
Rotate single right



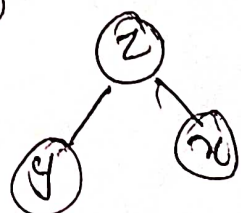
LR rotation



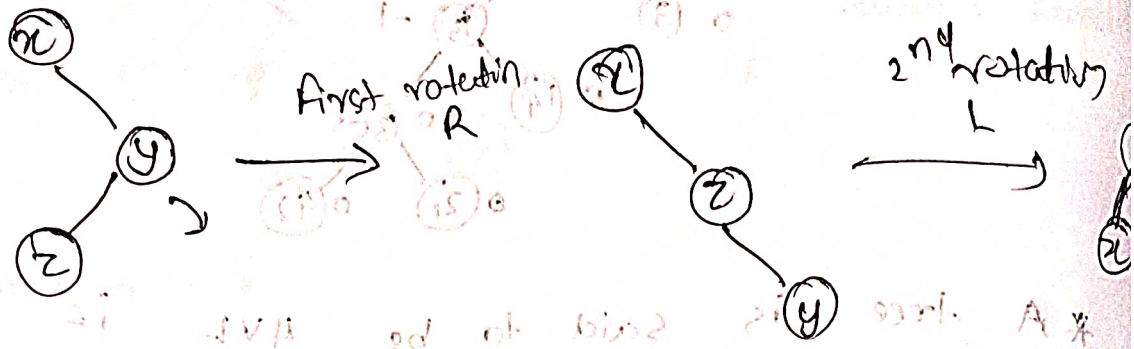
First rotation (L)



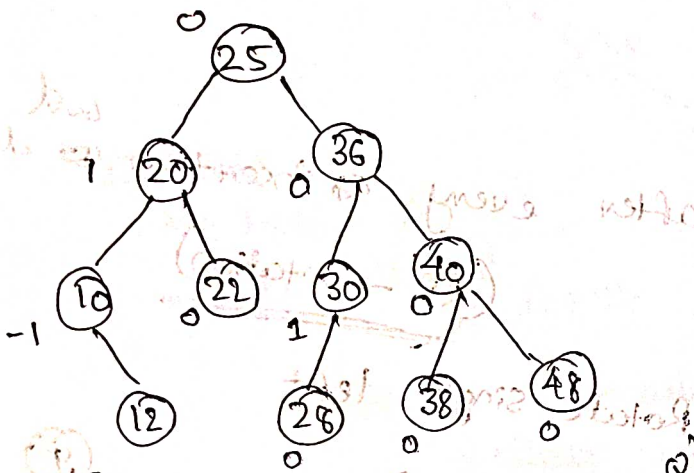
2nd rotation (R)



RL rotation



Q.



Construct an AVL tree by inserting the following

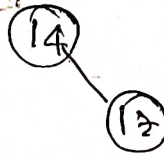
14, 17, 11, 7, 53, 4, 13, 12

(rotation 11)

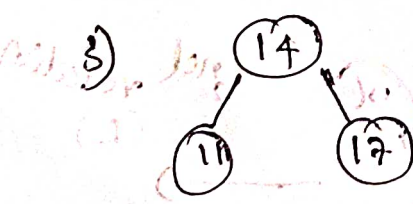


14

2)

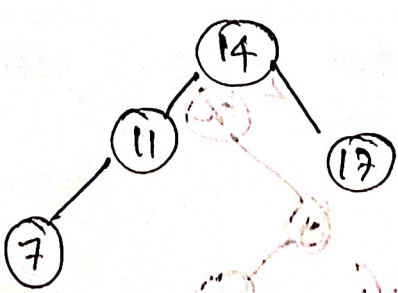


rotation 11

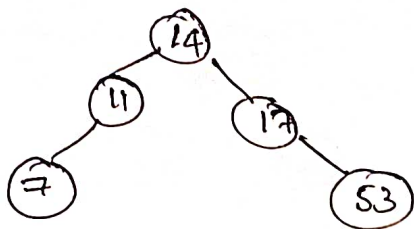


rotation 11

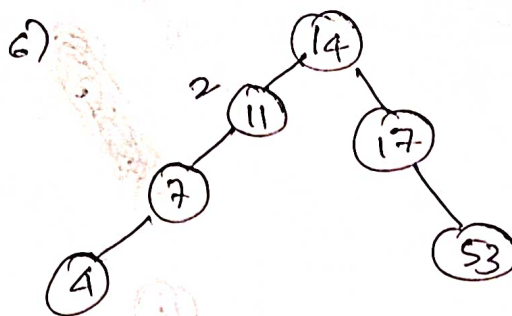
4)



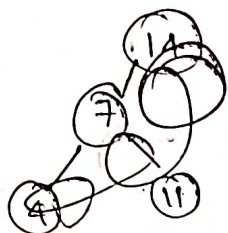
5)



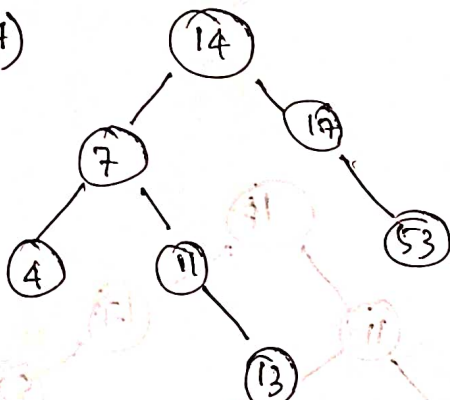
6)



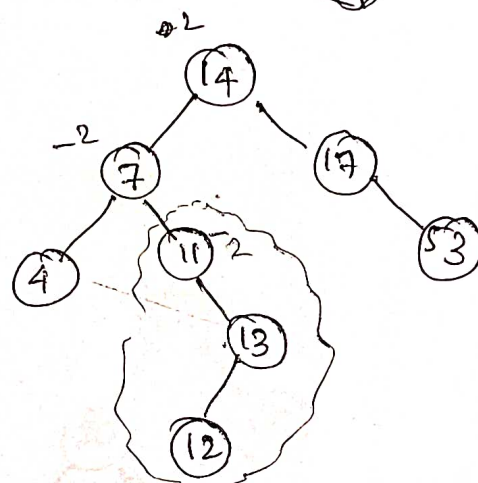
single right



7)



8)

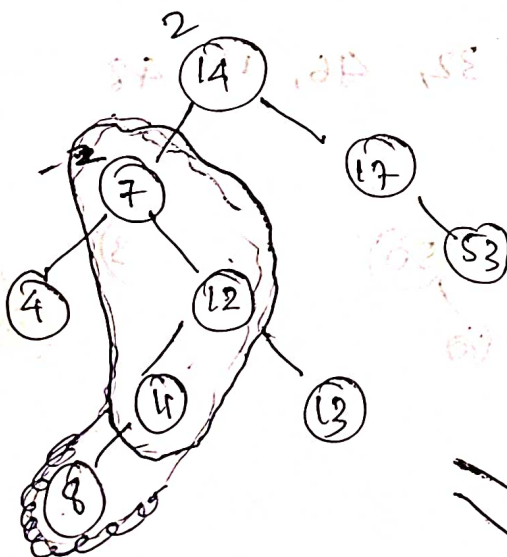


R

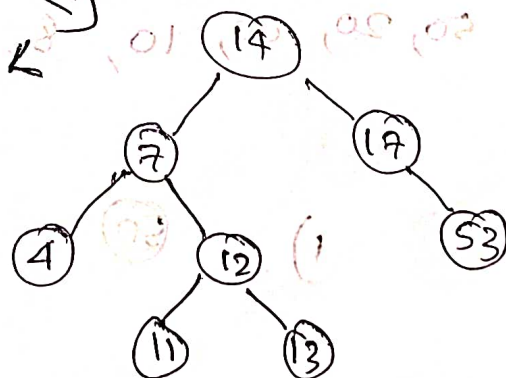


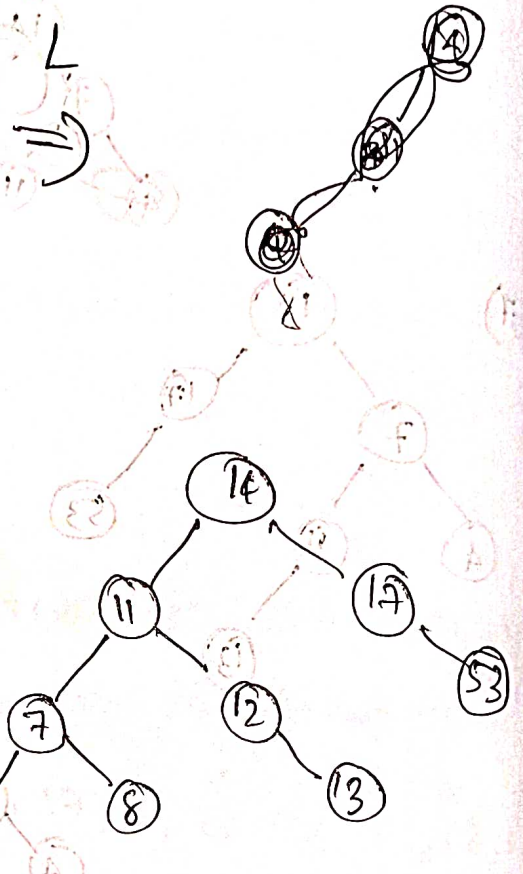
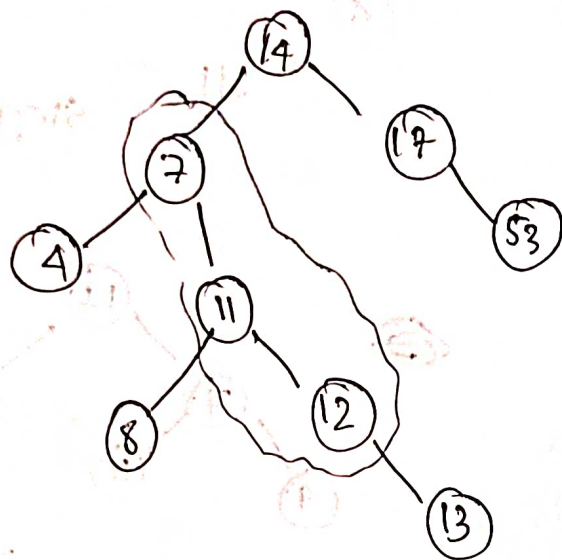
RL rotation

9)



R





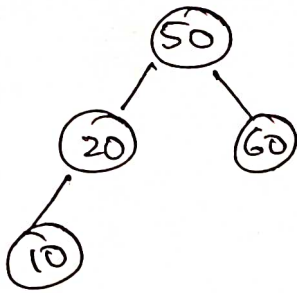
Q) 50, 20, 60, 10, 8, 15, ~~22~~ 32, 46, 11, 48

Ans) 1) 50

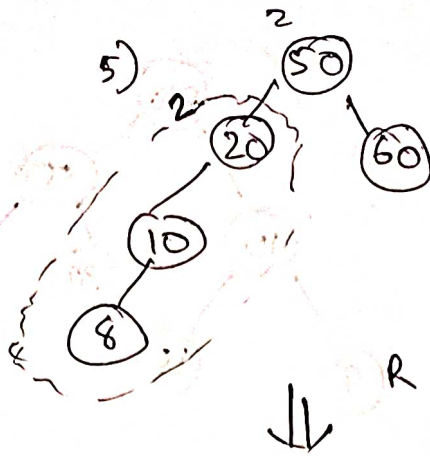
2) 50

3) 50

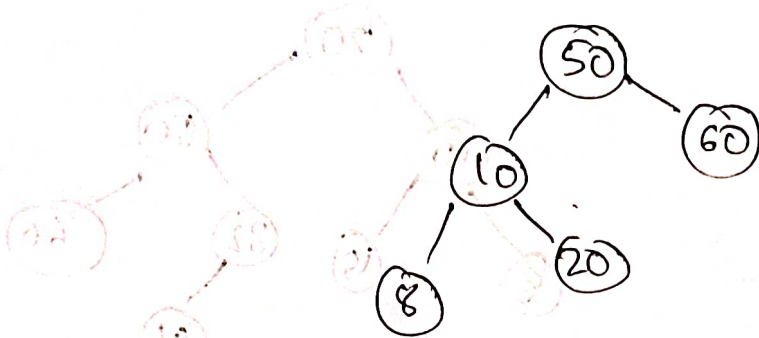
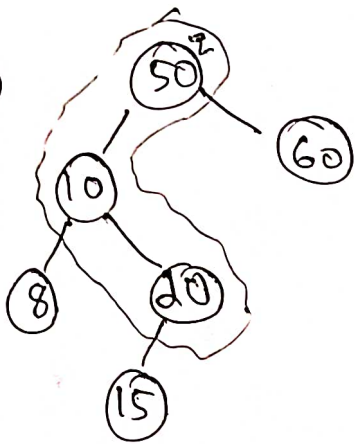
4)



5)

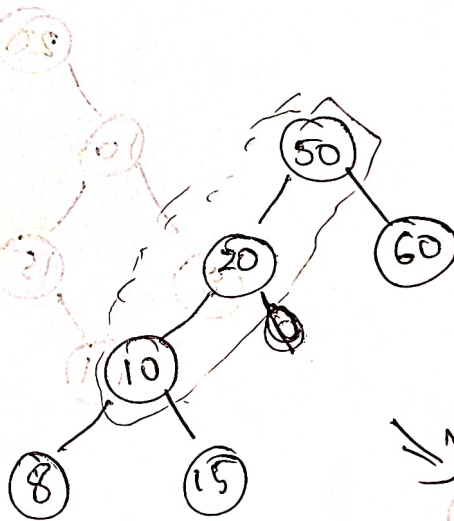
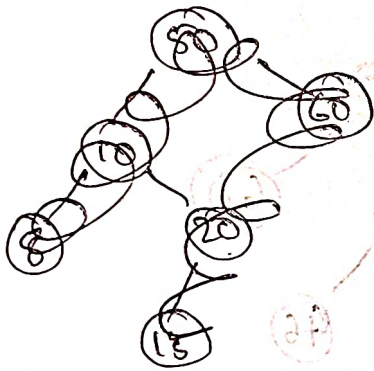


6)

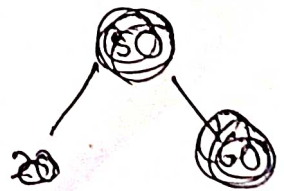
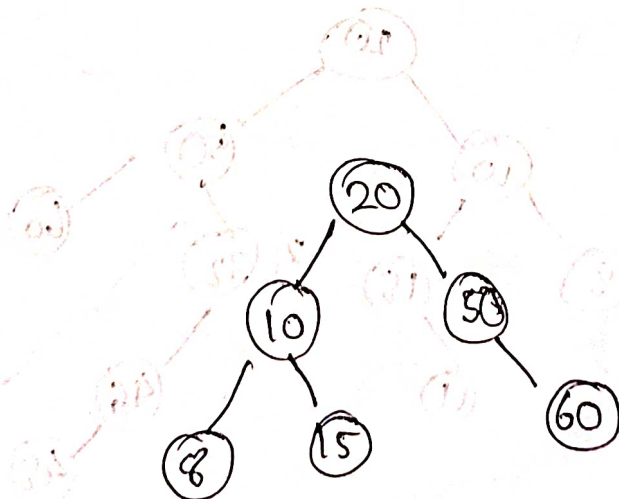


LR rotation
R

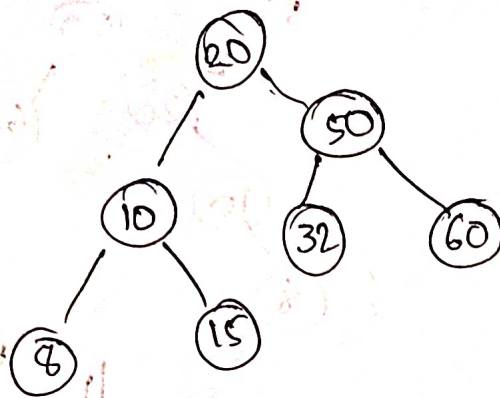
7)



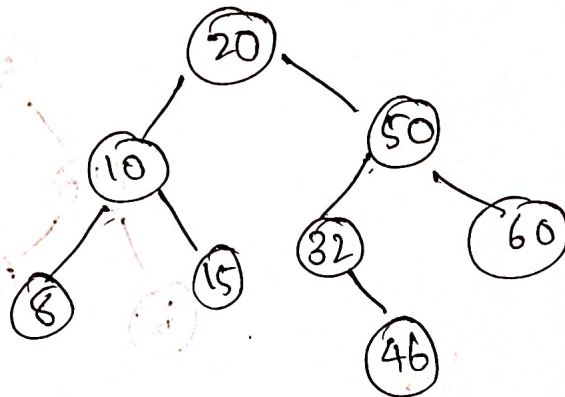
LR



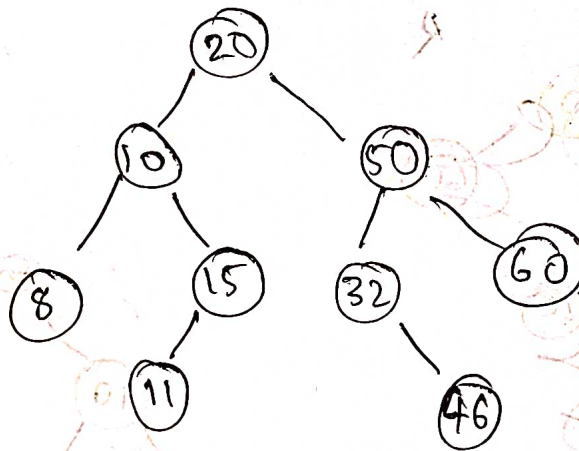
7)



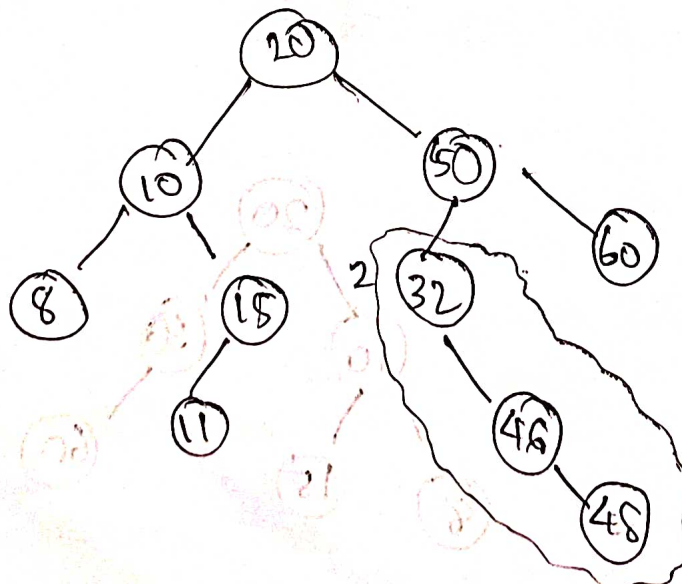
8)



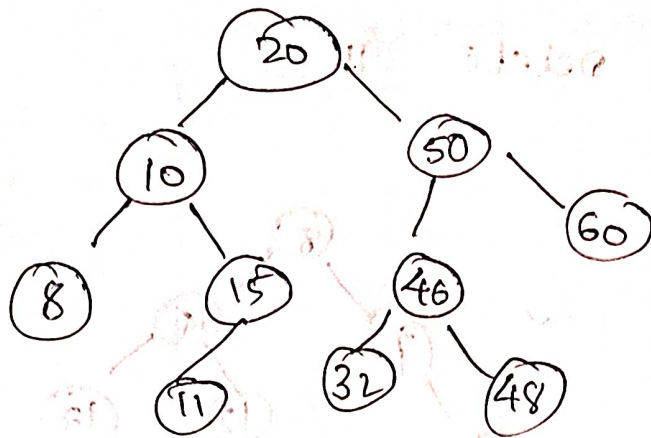
9)



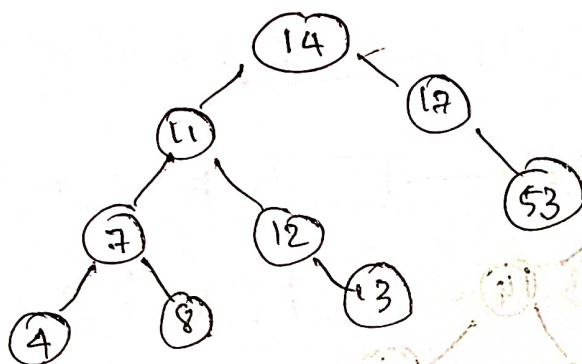
10)



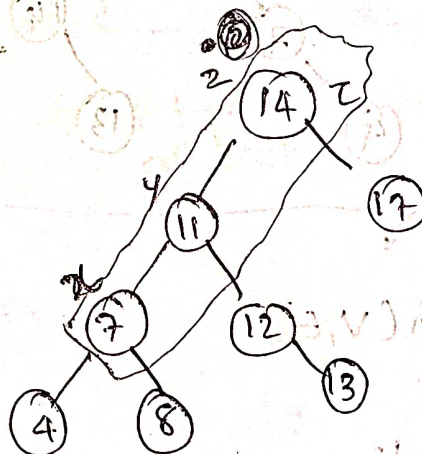
L $\Rightarrow$



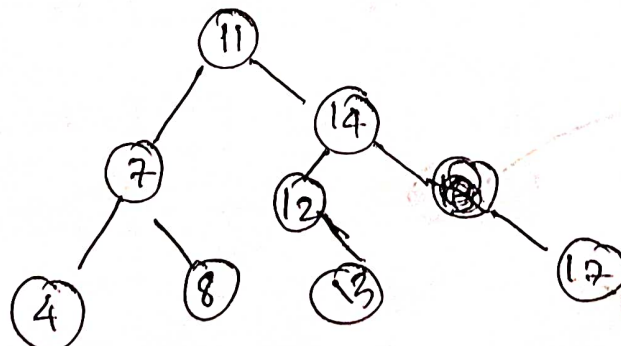
Q)



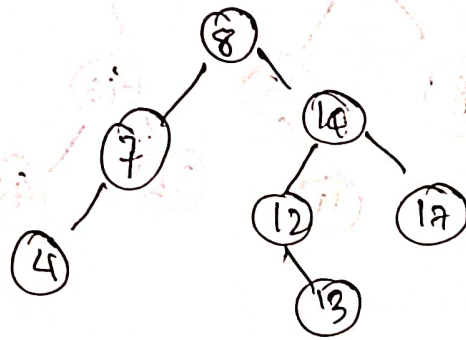
delete 53



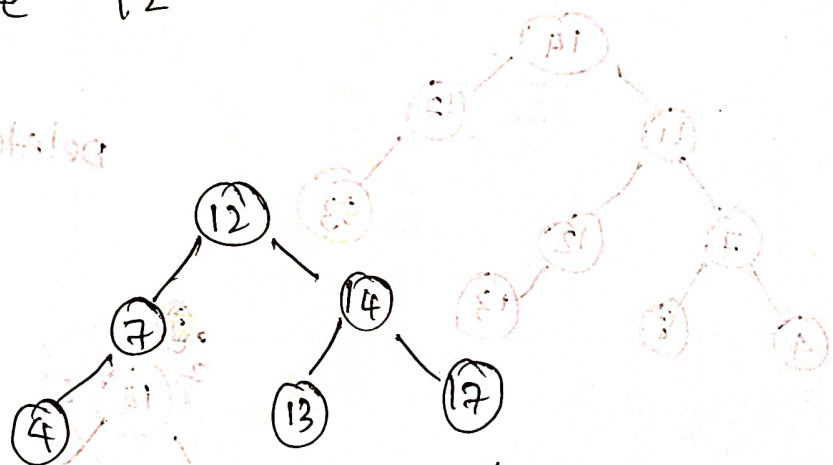
R $\Leftarrow$



Delete 11



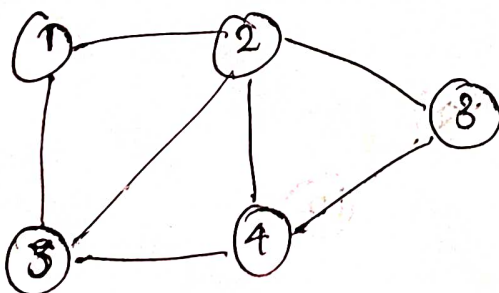
Delete 12



Graph $G(V, E)$

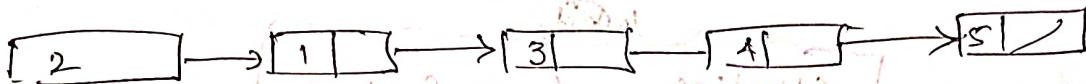
Graph representation

- Adjacency list
- Adjacency matrix

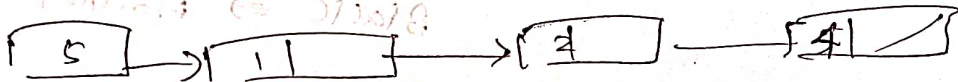
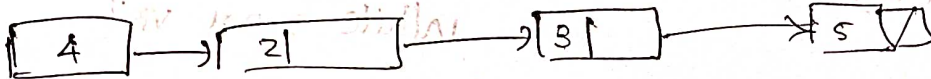
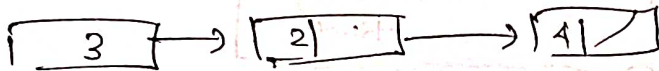


	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	0	0	1	0
4	0	1	1	0	1
5	1	0	0	1	0

matrix

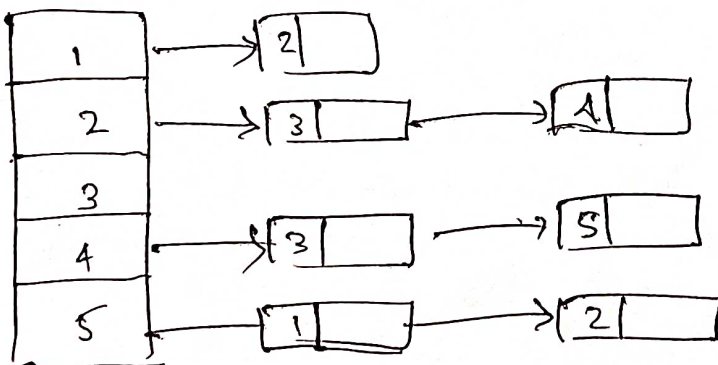
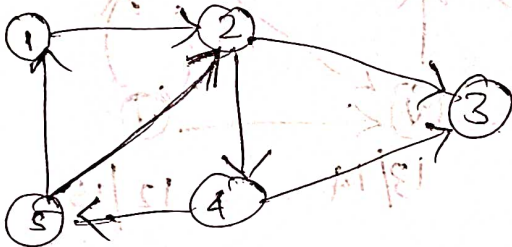


List



total length of list

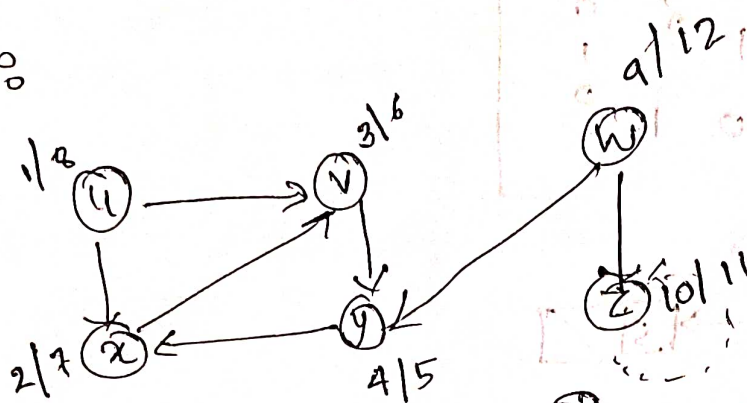
undirected = 21
= 14



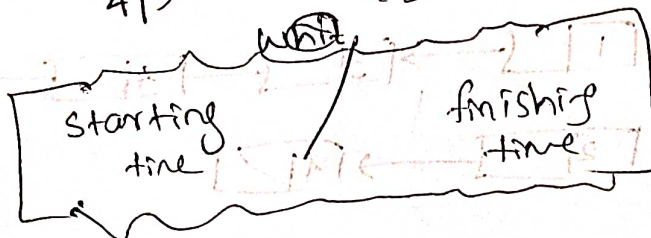
total length of list
directed = 6

BFS $\Rightarrow$ * time complexity $O(V+E)$ * problem
 Algorithm

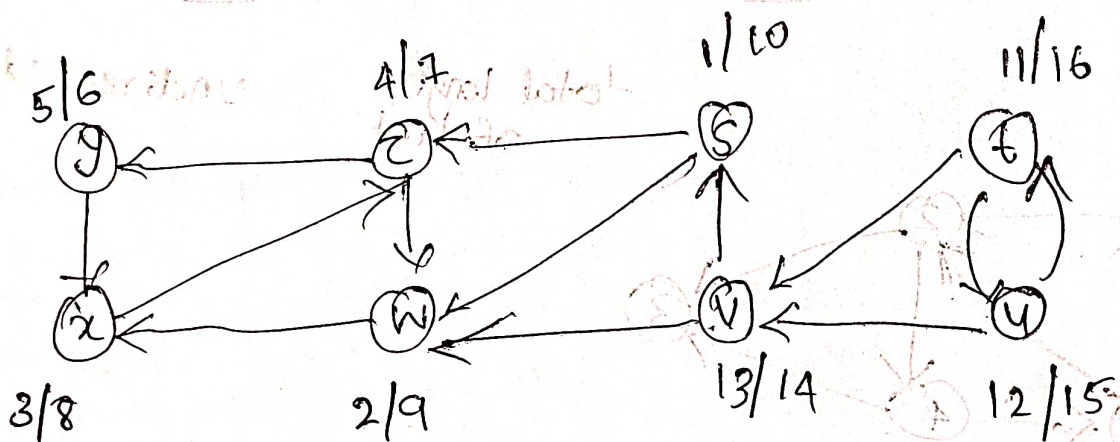
DFS



3 colors $\Rightarrow$
 (white, grey, black)



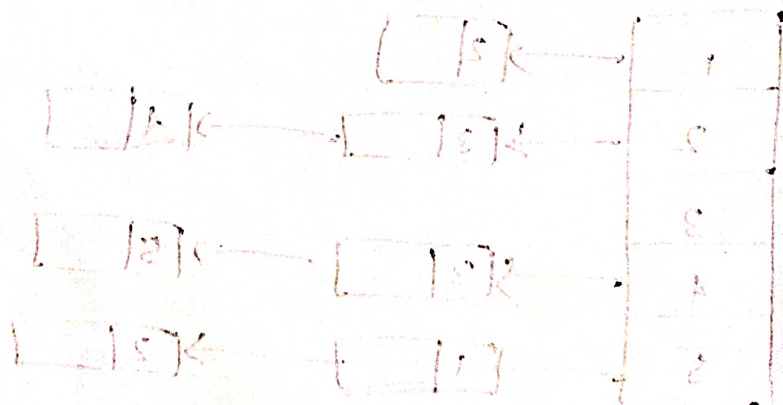
white $\rightarrow$ not visit
 Black $\Rightarrow$ finished



Pseudo code

* * *

DFS(u)



Classification of Edges

x — x — x — x

1. Tree edge $\Rightarrow (u,v) \Rightarrow$ all exploring edge
2. Back edge $\Rightarrow v$ is descendant
3. Forward edge u is descendant
4. Cross edge.

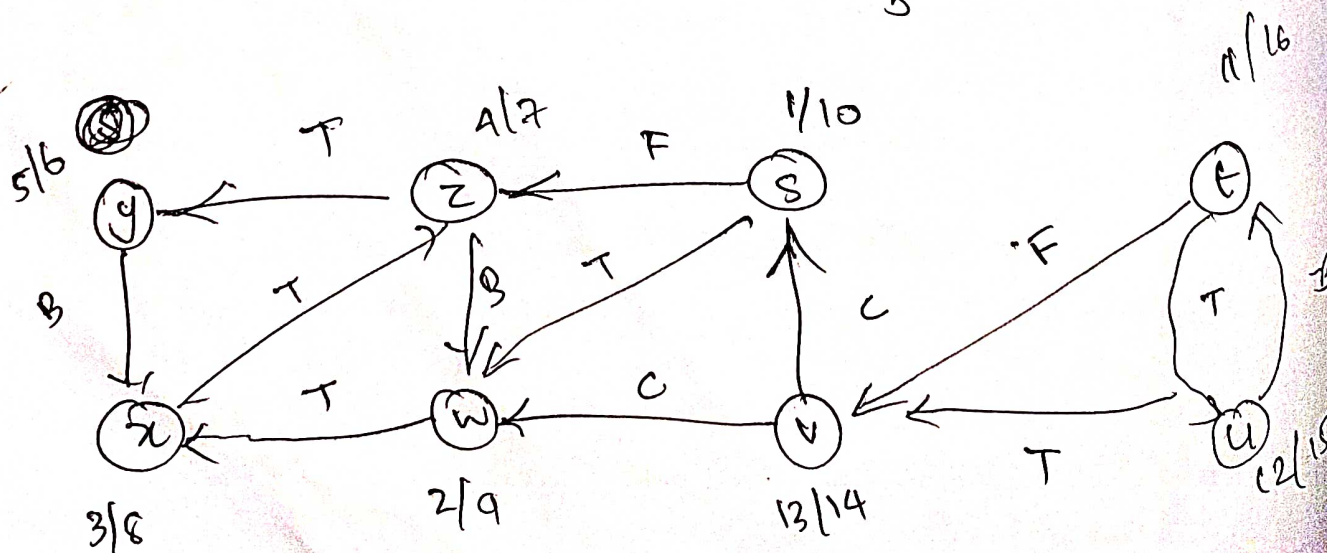
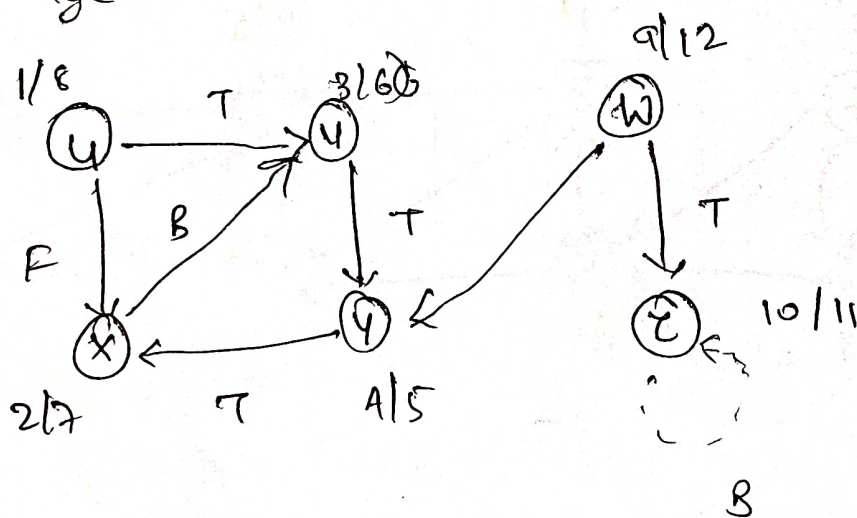


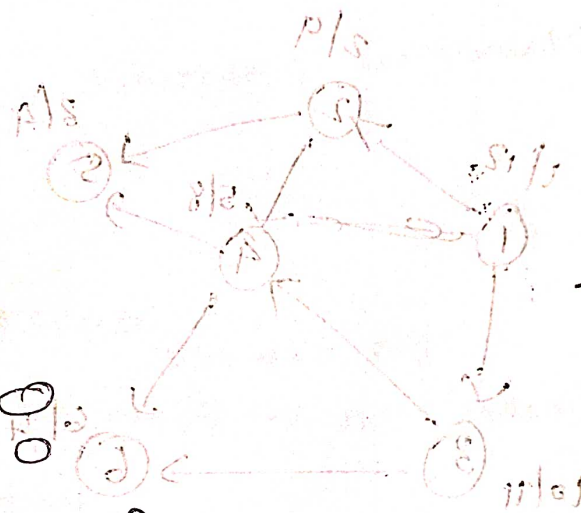
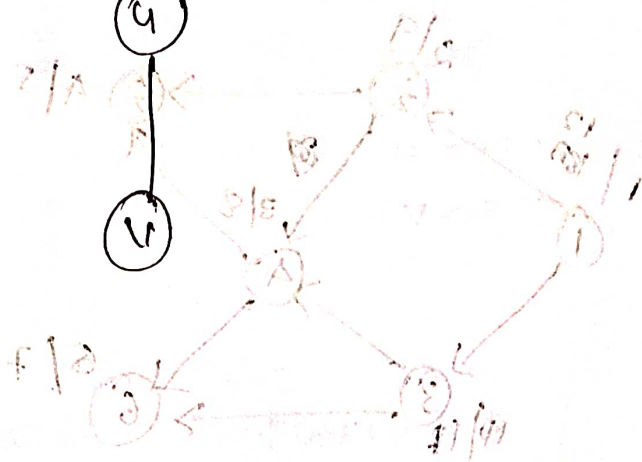
Tree edge = $(u,v), (v,y), (y,x), (w,z)$

Back edge = $(x,v) \Rightarrow$ ('loop edges')
 (z,z)

Forward edge = (y,x)

cross edge = (w,y)





Topological Sort

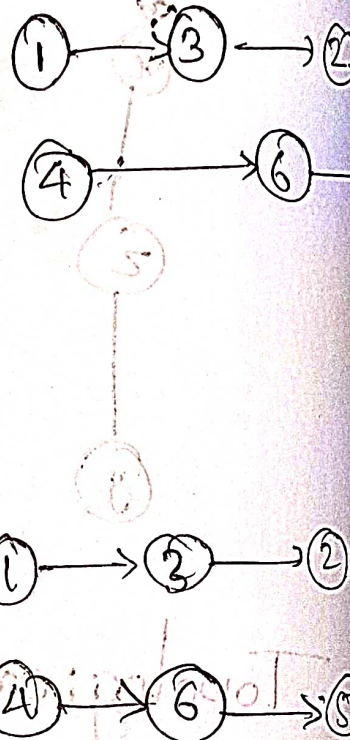
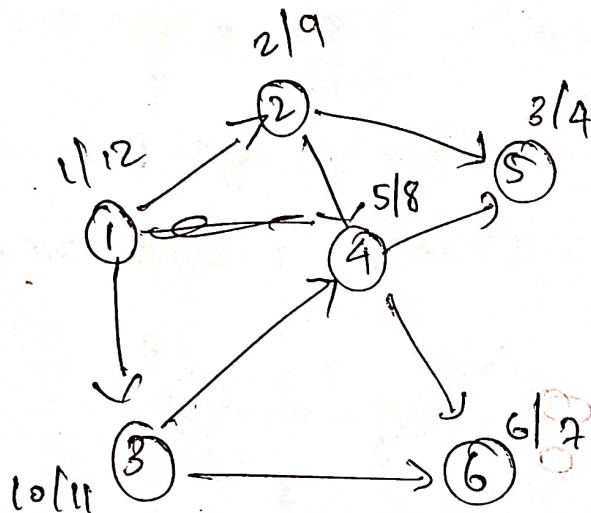
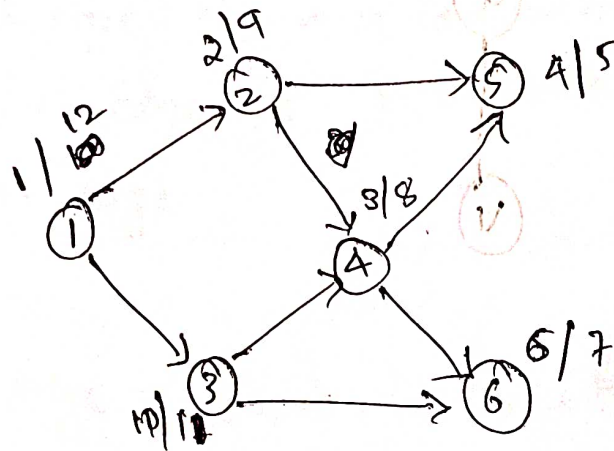
* Directed Acyclic graph (DAG)

* A topological sort of a DAG $G=(V,E)$ is a linear ordering of all its vertices such that if G contains an edge (u,v) , then u appears before v in the ordering.

Algorithm:

1. Call DFS(w) to compute finishing times $FC(u)$ for each vertex.
2. As each vertex is finished, insert it onto the

2. return the linked list of vertices.



(WAG). deqorp 210 p2A b420010 7

on edge (11,1) after a slight change in the
order of cells. The neutralization test is a constant
of a PAD is (3,4) is a constant

Handwritten signature: *Handwritten signature*

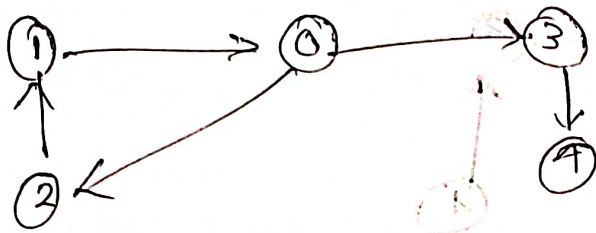
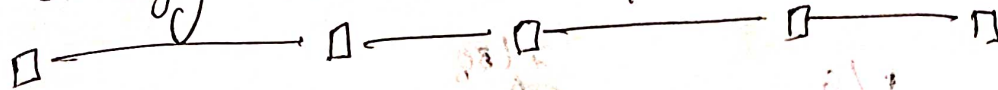
2004-05-18

COPIES OF THE REPORT OF THE

43434 1003

4 of 1990

Strongly Connected Components (SCC)



$\{0, 1, 2\}$ $\{3\}$ $\{4\}$

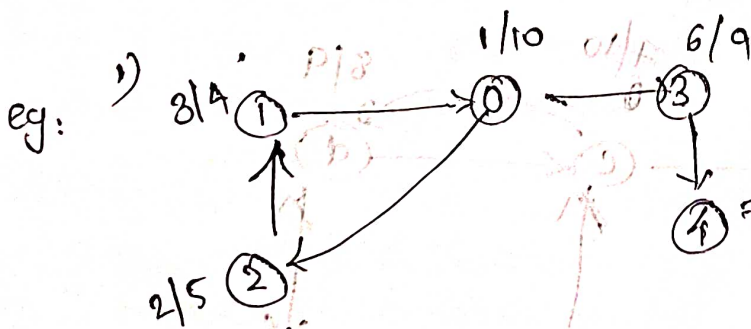
$u \leq v$
 $\downarrow$
 (u, v)
 $u \rightsquigarrow v$
 $v \rightsquigarrow u$

$\rightsquigarrow$: reach
 able

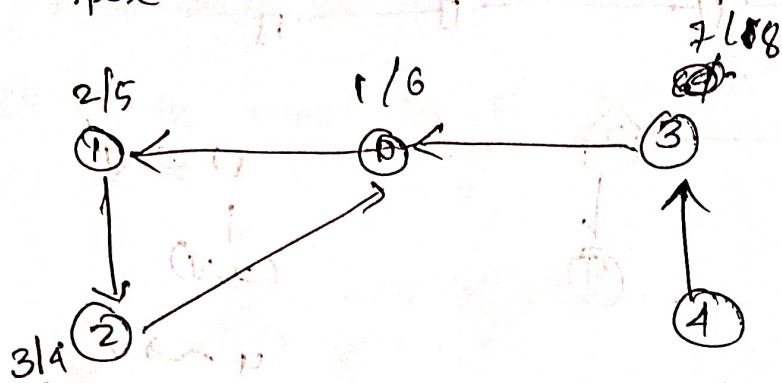
strongly connected components.

Algorithm to find Strongly connected components

1. perform DFS
2. construct graph transpose (reverse edge)
3. Again Perform DFS by considering the order of decreasing Finishing time
4. output the vertices of each tree in DF forest as strongly connected graph.



2) Transpose

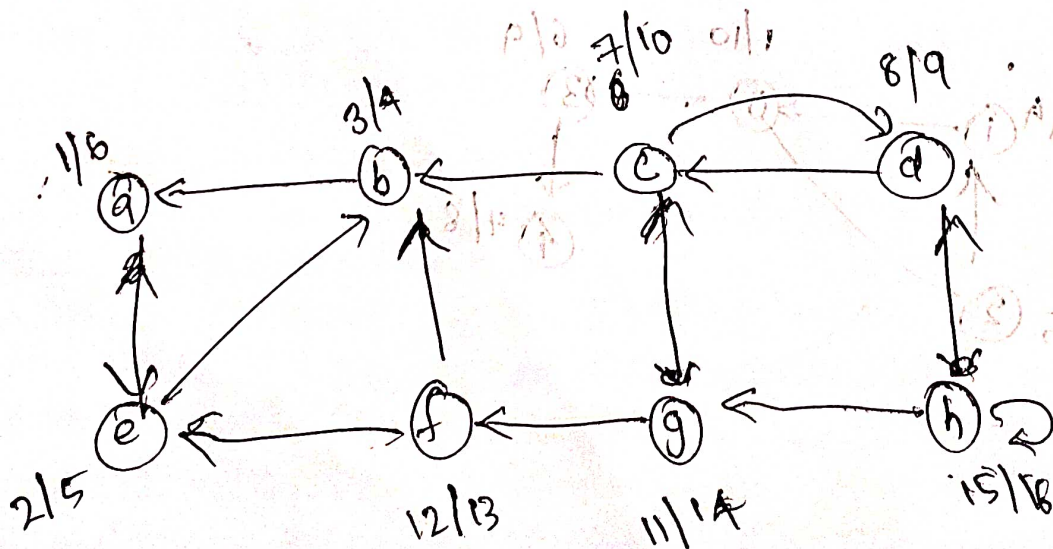
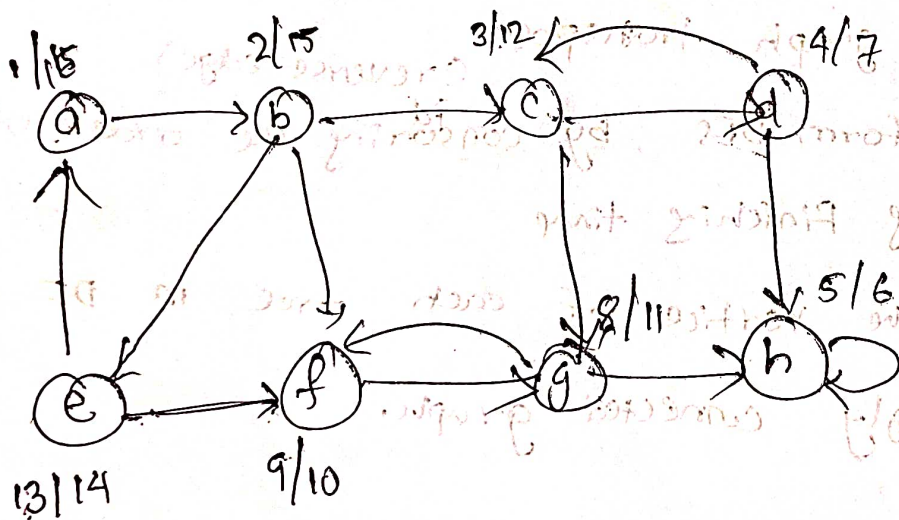


③

④

$\{0, 1, 2\}$, $\{3\}$, $\{4\}$

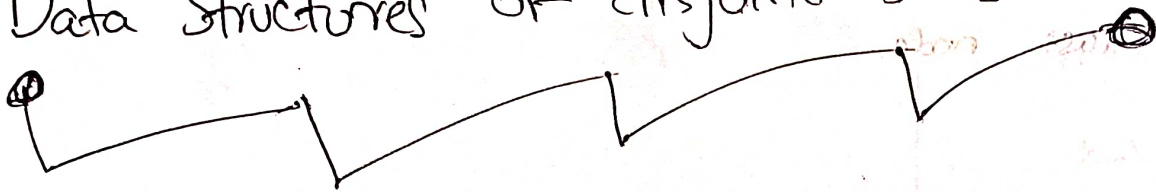
Q)





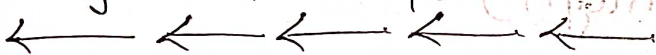
$\{a, e, b\}, \{c, d\}, \{g, f\}, \{h\}$

Data Structures of disjoint sets



Eg: Connected component from a graph.

Disjoint-Set operations



$x \rightarrow$ an object

1. MAKE-SET(x) : Create a new set with a single member pointed to by x .
2. UNION(x, y) : unites the dynamic sets that contain x and y , say S_x and S_y into a new set.
3. FIND-SET(x) : returns a pointer to a representative of the set containing x .

Analysis

* Amortized Analysis

n : no. of make-SET operations

m : total no. of make-SET, UNION, and FIND-SET operations

$m \geq n$

* It calculate the time complexity of series of operation.
[Amortized analysis]

2 type implementation

1 → Linked list implementation.

• Every disjoint set is a linked list;

* Back pointer $\Rightarrow$ point to the first node.

* representative ~~node~~ has a pointer which point last node

Union

* weighted union heuristic: append shorter list to longer.

* Total time: $O(m + n \log n)$

2. → Disjoint Set forest implementation

Represented as tree $\Rightarrow$ every disjoint set

Representative $\Rightarrow$ root node.

each node points to its parents

operations

~~make sets~~

MAKE-SET: Create a tree with one node

FIND-PATH: follow parent pointer until we get

root

UNION: root of one tree points to root of other.

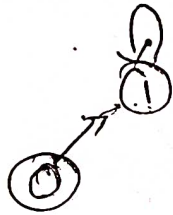
Naive Union

for $i = 0$ to 3

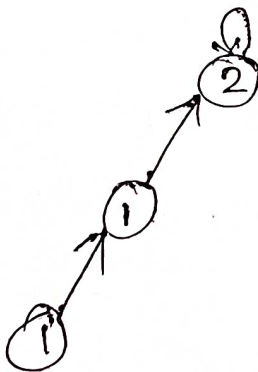
* make SET (i)



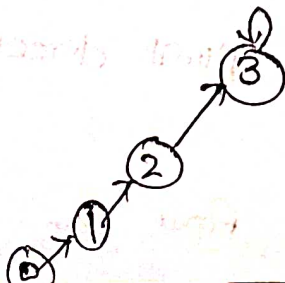
• union (0, 1)



• union (1, 2)



• union (2, 3)



Algorithm

1- UNION (x, y)

$x-1 = \text{FIND-SET}(x)$

$y-1 = \text{FIND-SET}(y)$

~~LINK $(x-1, y-1)$~~

$P(x-1) = x-1$

MAKE-SET (x)

$P(x) = x$

$\text{rank}[x] = 0$

Union by Rank



* Improve the performance of union-set

rank = approximately the height

UNION(x, y):

x-1 = FINDSET(x)

y-1 = FINDSET(y)

LINK(x-1, y-1)

LINK(x-1, y-1):

(x) T32-2444

(y) T32-2444

(x) T32-2444

1-x = (1-x)9

(x) T32-2444

x = (x)9

0 = (x)9

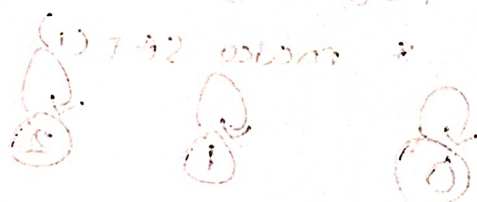
Path compression



* Improve the performance of find-set

* make each node on the find path point directly to the root

* The idea is to flatten the tree find set



(1,0) not a



(5,1) not a

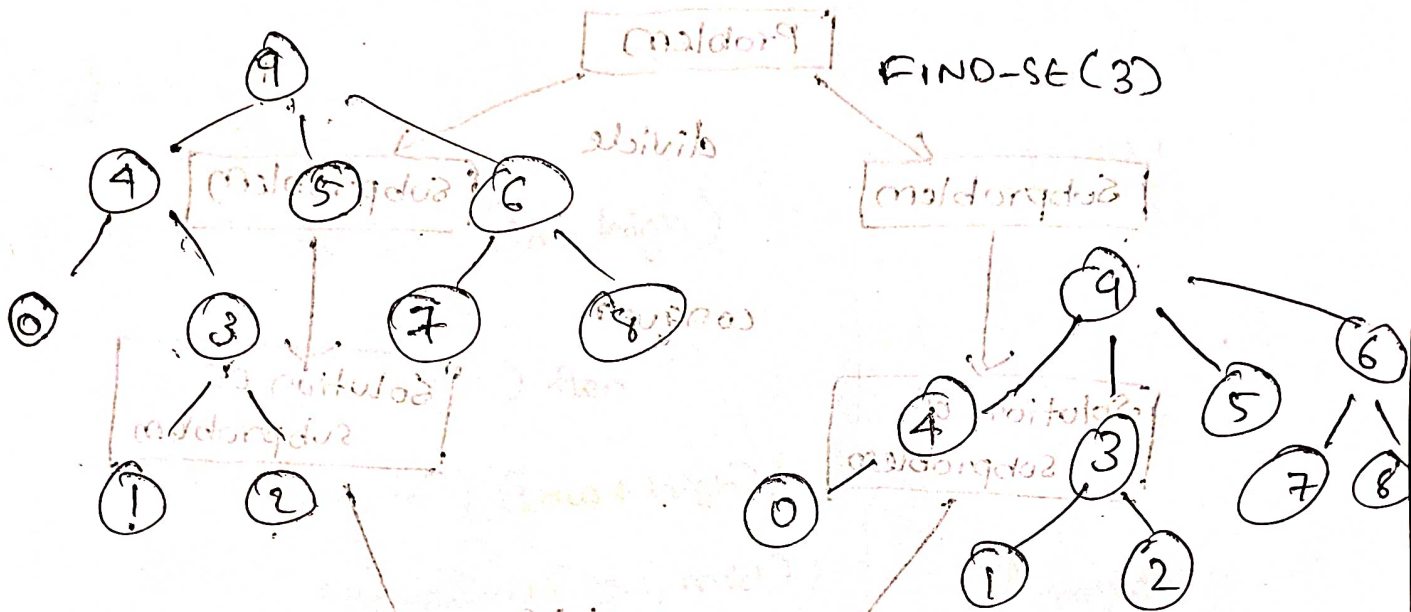


FIND-SET(x)

if $x \neq P(x)$

$P(x) = \text{FIND-SET}(P(x))$

return $P(x)$



Time complexity $\alpha(n)$

m operations
 n disjoint

Algorithm Disjoint

if $(x) \neq P(x)$ then return $P(x)$

end